

2026 春《C 语言课程设计》日志（二）

题目：文具店销售管理系统 组号：9 日期：2026 年 6 月 1 日 得分（满分 10 分）：

日志内容：

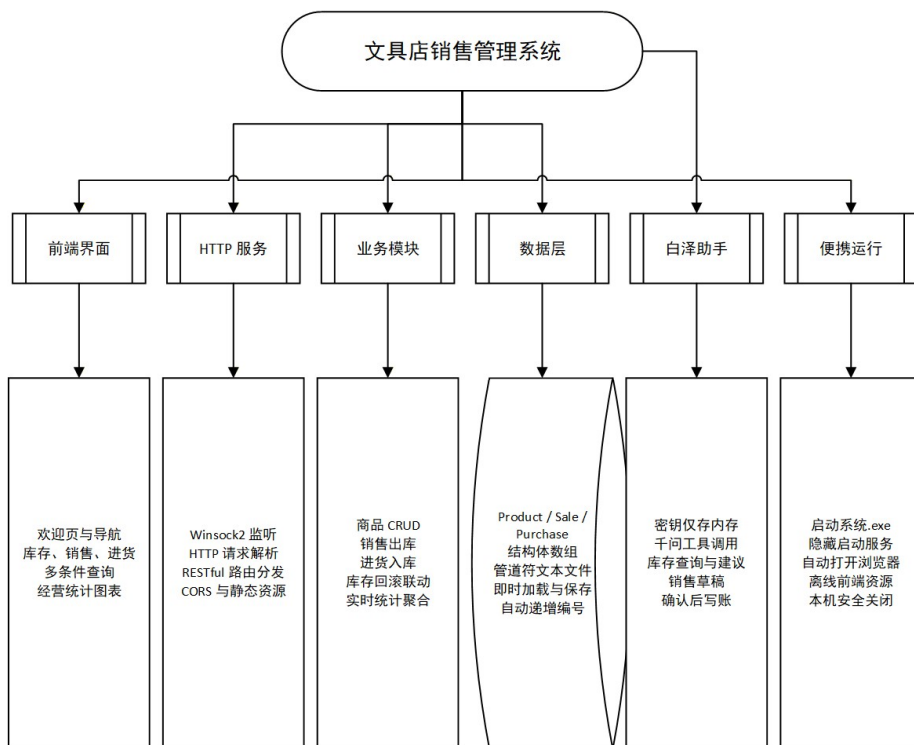
1. 系统功能结构。画出系统功能结构图，介绍系统核心功能模块。
 2. 详细说明各模块的功能、输入、输出、处理过程，画出各模块的流程图。
 3. 系统的软、硬件开发环境简介，系统相关技术或算法简介。
 4. 系统的设计与实现主要用到 C 语言哪些知识点？详细说明这些知识点的语法。
- 要求：不少于 2000 字。

1 系统功能结构

本项目在日志（一）的需求分析基础上完成了可运行的文具店销售管理系统。系统面向中小型文具店的日常进销存场景，以商品库存为中心，把销售出库、进货入库、多条件查询、经营统计和辅助决策连接成一套完整流程。后端坚持使用 C 语言实现，不依赖 Web 后端框架；前端则采用本地离线资源加载方式，在浏览器中提供可视化操作界面。

系统总体上划分为前端界面、HTTP 服务、业务模块、数据层、白泽助手和便携运行六个部分。各部分通过清晰的接口协作：浏览器将请求发送到本机 C 服务，服务器完成协议解析和路由分发，业务模块修改结构体数组并写回文本文件，前端再根据 JSON 响应刷新界面。图 1 展示了当前版本的功能结构。

图 1 文具店销售管理系统功能结构图



2026 春《C 语言课程设计》日志（二）

1.1 前端界面与交互层

前端页面承担信息展示和操作入口。当前实现包含欢迎页、库存管理、交易记录、进货记录、新建销售、录入进货、多条件查询、经营统计和使用说明等页面。库存、销售和进货列表支持分页、筛选、排序、修改和删除；销售与进货录入页会在提交前实时预览数量、金额和变动后的库存。经营统计页展示近 7 天销售趋势、品类销售额占比、商品销售额排行以及全商品明细。

日志（一）中计划使用 Vue 3 CDN 模式。实际实现阶段根据便携交付目标调整为 React、ReactDOM、Babel 和 GSAP 的本地离线资源模式。这样既保留了组件化界面的可维护性，又避免了演示时依赖公网。

1.2 C 语言 HTTP 服务层

HTTP 服务由 server.c 实现，使用 Winsock2 在 8080 端口监听本机请求。服务器既提供前端静态资源，也提供 RESTful 风格的数据接口。主要路由包括 /api/products、/api/sales、/api/purchases、/api/stats/*、/api/assistant/* 和 /api/system/shutdown。服务端根据请求路径和 HTTP 方法调用对应处理函数，并统一返回 JSON 响应。

1.3 业务模块与数据层

业务层由 product.c、sale.c、purchase.c、stats.c 等模块组成。商品模块负责商品资料的增删改查；销售模块负责销售记录与库存扣减；进货模块负责入库记录与库存增加；统计模块在内存数组上实时聚合经营指标。数据层由 data.c 和 data.h 组成，定义 Product、Sale、Purchase 三种结构体，并负责文本文件加载、保存、编号生成和按商品编号查询。

1.4 白泽助手与便携运行

项目在基础进销存之上增加了白泽智能助手。助手后端仍然使用 C 语言，通过 WinHTTP 调用千问模型，并限制模型只能调用系统提供的白名单工具。涉及销售时，助手只能先生成草稿，必须由用户点击确认后才能写入账册。便携运行部分则通过“启动系统.exe”静默启动后端并自动打开浏览器，网页中还提供本机安全关闭入口。

2 核心模块设计与实现

2.1 数据层模块

2026 春《C 语言课程设计》日志（二）

功能：数据层负责统一保存商品、销售和进货三类数据。输入为文本文件中的管道符分隔记录，输出为内存中的结构体数组；当业务模块发生增删改时，数据层再将数组写回对应文件。

处理过程：程序启动后依次调用 `load_products()`、`load_sales()` 和 `load_purchases()`。每个加载函数使用 `fopen()` 打开文件，跳过表头后通过 `fgets()` 逐行读取，再利用 `strtok()` 按“|”切分字段。保存时使用 `fprintf()` 按固定字段顺序逐行写出。商品编号采用 P001，销售编号采用 S001，进货编号采用 B001。生成新编号时，不直接使用数组长度，而是遍历已有编号找到最大数值后加一，从而避免删除记录后出现编号冲突。

该实现适合课程规模：数据直观、调试方便、无需第三方数据库。需要说明的是，文本文件方案可以做到“先校验、后修改、立即写回”，但尚未提供数据库事务或日志恢复能力。若发生突然断电，理论上仍存在文件写入中断风险。后续迁移到 SQLite 时，可使用事务进一步提高可靠性。

2.2 商品库存管理模块

商品模块的输入包括商品名称、类别、生产厂家、型号、库存数量和单价，输出为商品列表或操作结果 JSON。创建商品时，系统生成新的 P 编号并保存；修改商品时，只更新请求体中实际出现的字段；删除商品时，使用数组元素前移覆盖的方式移除目标记录。列表查询支持类别精确筛选和关键字模糊匹配，关键字可匹配商品编号、名称和厂家。

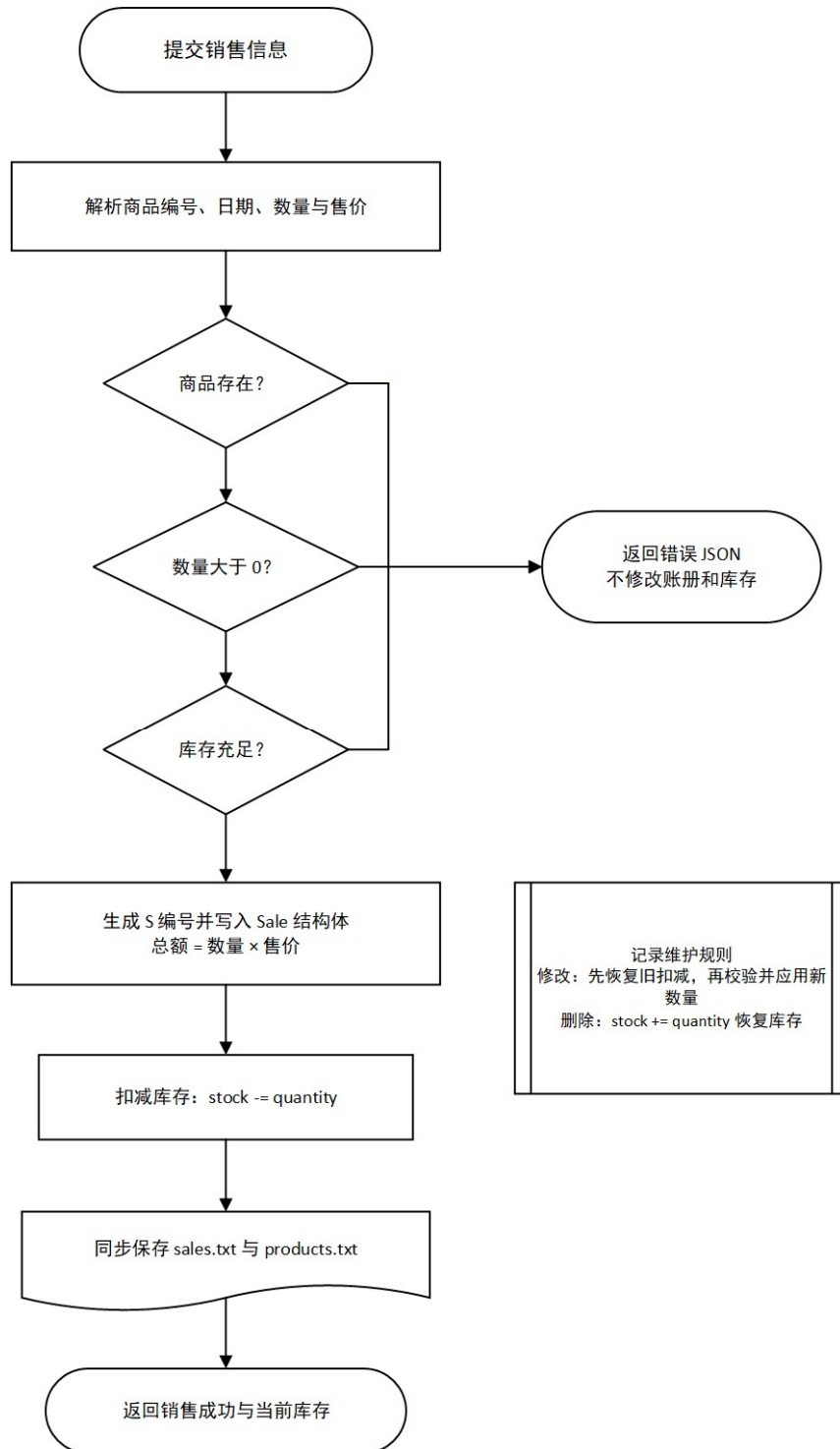
商品对象是系统的基础数据。销售和进货模块都通过 `find_product_by_id()` 找到对应商品，读取名称、类别和当前库存。统计模块也以商品数组为起点，计算库存总量、库存总价值以及低库存预警数量。

2.3 销售与进货库存联动模块

销售和进货是本系统最重要的业务联动。销售记录的输入为商品编号、交易日期、销售数量和实际售价，输出为销售编号、交易金额和变更后的库存。创建销售前，系统首先检查商品是否存在、数量是否大于 0、当前库存是否充足；全部校验通过后才生成 S 编号、写入 Sale 结构体、执行 `stock -= quantity`，并同步保存 `sales.txt` 与 `products.txt`。图 2 展示了销售创建的处理流程。

2026 春《C 语言课程设计》日志（二）

图 2 销售创建与库存联动流程图



销售记录修改时采用“恢复旧扣减 → 校验新数量 → 应用新扣减”的思路，删除销售记录则执行 $\text{stock} += \text{quantity}$ ，以模拟退货后的库存恢复。进货模块与销售模块方向相反：新增进货执行 $\text{stock} += \text{quantity}$ ，删除进货执行 $\text{stock} -= \text{quantity}$ ，并在极端情况下将库存下限限制为 0。两个模块形成对称关系，使账册记录与库存数量保持联动。

2026 春《C 语言课程设计》日志（二）

2.4 多条件查询与经营统计模块

查询模块允许在库存、销售和进货三个数据集上组合筛选商品编号、名称、类别、厂家、日期区间、数量区间和价格区间。统计模块则直接遍历内存数组实时计算，不额外维护汇总表。对于课程设计中的百条级数据，这种实现简单且足够快速。

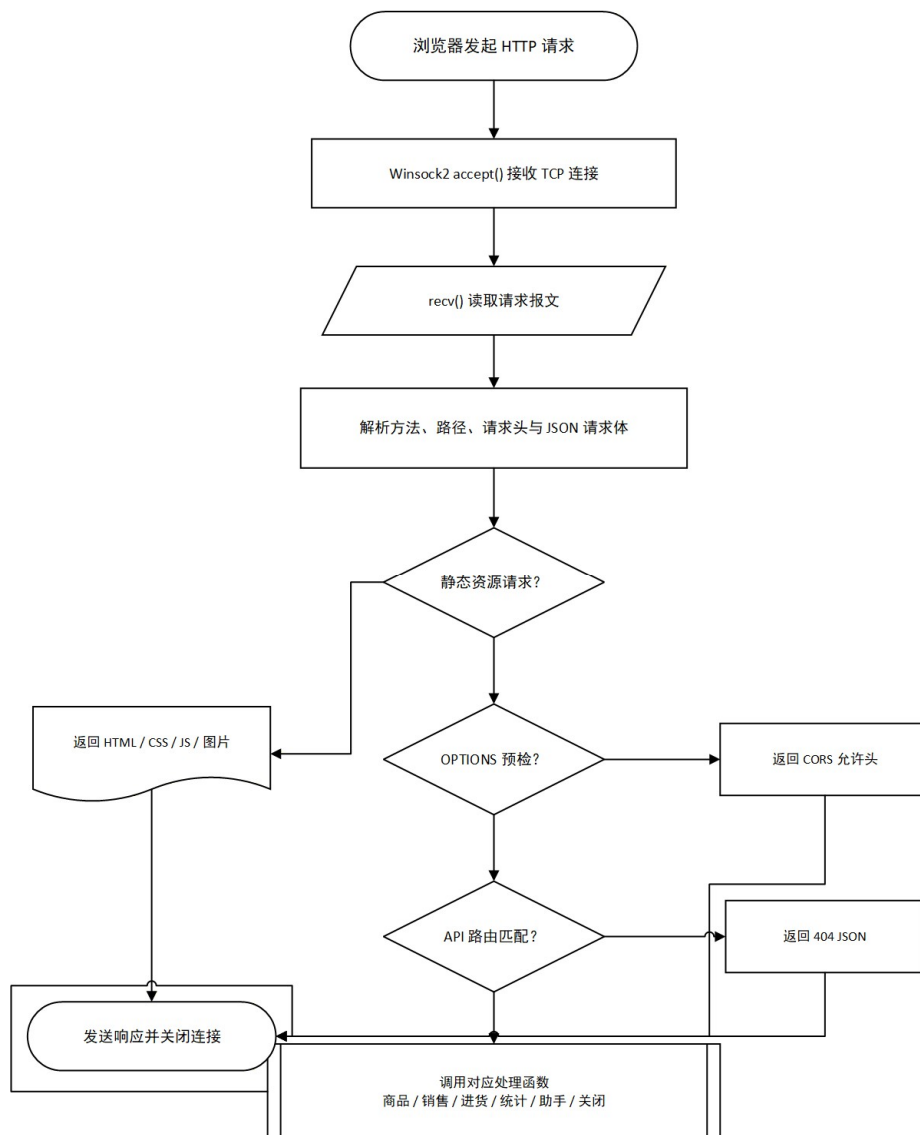
统计接口包括：仪表盘综合指标、销售排行榜、低库存预警、近 7 天品类销售统计和库存全景报告。仪表盘汇总商品数量、总库存、库存总价值、历史销售额、历史进货额、简化利润、今日销售额、本周销售额和本月销售额；排行榜按商品累计销售金额降序排列；库存报告对每个商品汇总历史销售量、销售额、进货量和进货额。前端基于这些接口绘制柱状图、环形图和排行条。

2.5 HTTP 请求处理与本机关闭

浏览器访问系统后，服务器使用 `accept()` 接收 TCP 连接，再通过 `recv()` 读取请求报文。服务器解析请求方法、路径、请求头和 JSON 请求体。如果请求目标是静态资源，则返回 HTML、CSS、JavaScript 或图片；如果是 OPTIONS 预检请求，则返回 CORS 允许头；如果是 API 请求，则分发给对应业务函数；无法匹配时返回 404 JSON。处理结束后发送响应并关闭本次连接。图 3 展示了该流程。

图 3 C 语言 HTTP 请求处理流程图

2026 春《C 语言课程设计》日志（二）



服务器采用单线程、短连接模型：每次处理一个请求，发送响应后关闭连接，再等待下一次连接。该模型适合单机演示，代码结构清晰。系统关闭接口 `/api/system/shutdown` 只允许 `localhost` 调用，避免局域网其他设备误关服务。关闭时还会清除白泽助手保存在内存中的密钥。

2.6 白泽助手模块

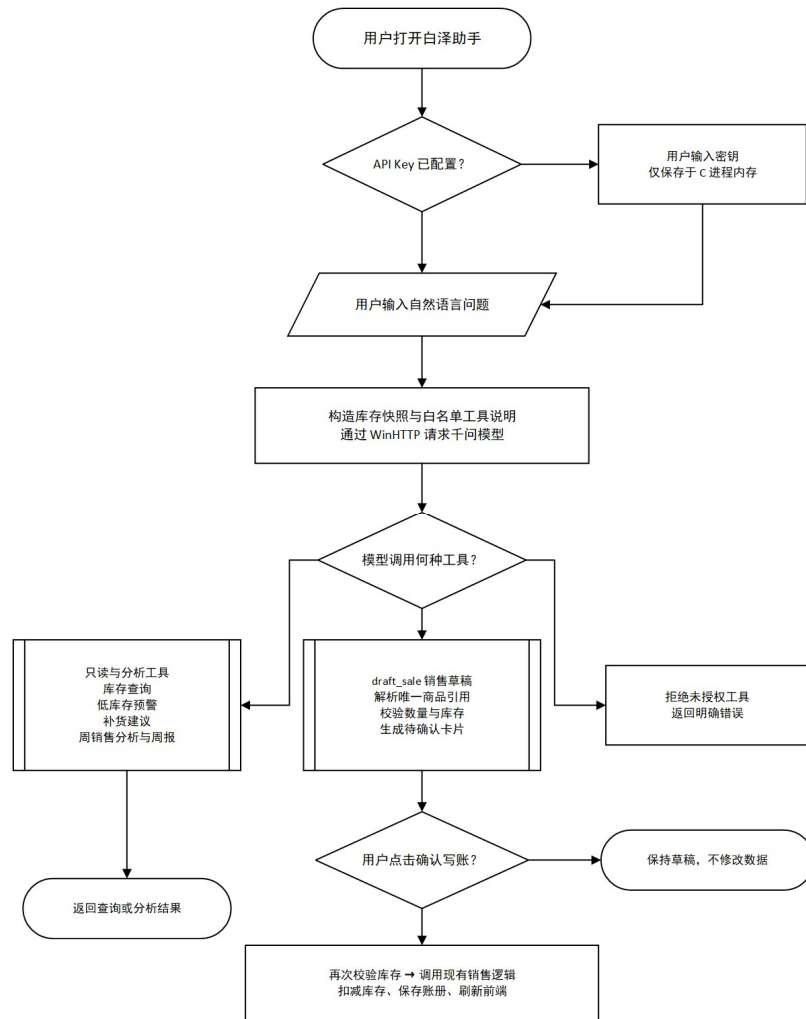
白泽助手由 `assistant.c`、`assistant.h` 和前端 `assistant.jsx` 共同实现。用户首次打开助手时输入阿里云百炼 API Key，密钥只保存在当前 C 服务进程的内存中，不写入代码、文件或浏览器存储。服务关闭后通过 `SecureZeroMemory()` 清除密钥。

助手向模型提供的能力不是任意文件访问，而是固定白名单工具：库存查询、低库存预警、补货建议、近一周销售分析、经营周报和销售草稿。模型若请求未授权工具，C 后端会拒绝执行。

2026 春《C 语言课程设计》日志（二）

销售请求采用两阶段流程：模型先调用 `draft_sale` 生成待确认卡片；用户点击确认后，后端再次检查库存，再调用现有销售逻辑扣减库存并保存账册。图 4 展示了这一安全边界。

图 4 白泽助手受控工具调用与安全写账流程图



商品引用解析支持编号、精确名称和带装饰性文本的唯一匹配。例如模型返回“P002 真彩中性笔”时，系统仍可定位商品；若文本同时命中多个商品，则拒绝猜测并返回错误。当前多商品确认会在写账前统一检查库存，但文本文件保存仍不具备数据库事务能力，这也是后续可继续增强的部分。

3 软件与硬件开发环境

系统定位为 Windows 平台的便携式课程设计项目，对硬件要求较低。普通个人电脑即可完成

2026 春《C 语言课程设计》日志（二）

开发和演示。开发与运行环境如下表所示。

类别	环境或工具	说明
硬件环境	普通 x64 Windows 个人电脑	建议具备 4 GB 及以上内存，可运行浏览器和本机服务
操作系统	Windows 10 / Windows 11	后端使用 Winsock2、WinHTTP 和 Windows 启动器接口
后端语言	C 语言	核心服务器、数据层、业务层、统计层和助手层均使用 C 实现
编译工具	GCC	通过 build_portable.bat 或 backend/build.bat 重新编译
前端技术	HTML、CSS、React、Babel、GSAP	资源存放于 frontend/vendor，可离线加载
数据存储	管道符分隔文本文件	products.txt、sales.txt、purchases.txt 位于 data 文件夹
运行入口	启动系统.exe	隐藏启动 backend/server.exe，并自动打开默认浏览器

4 相关技术与算法

4.1 结构体数组与文本持久化

系统使用结构体数组保存运行时数据。Product、Sale 和 Purchase 分别描述商品、销售和进货记录，字段与文本文件列一一对应。数组容量由 MAX_PRODUCTS、MAX_SALES 和 MAX_PURCHASES 常量限制，既便于控制内存，也便于在调试时观察数据。每次业务操作完成后立即保存文件，降低退出程序造成数据丢失的概率。

4.2 轻量级 JSON 构建器

前后端通信使用 JSON。由于 C 标准库没有原生 JSON 序列化功能，项目在 utils.c 中实现 JsonBuilder。构建器维护缓冲区、容量、当前长度和 first 标记位；写入字段时自动处理逗号、引号和字符串转义；同时支持对象和数组嵌套。这样可以避免在每个业务函数中重复手工拼接 JSON。

4.3 路由匹配与 CORS

服务器使用 path_match() 判断路径前缀，并在匹配时保证后续字符是结尾、查询参数或子路径分隔符，防止错误匹配。静态路由要优先于通用路由，例如 /api/products/stats 必须在 /api/products/:id 之前判断，否则 stats 会被误认为商品编号。所有 API 响应加入 CORS 头，使浏览器前端能够访问本机接口。

2026 春《C 语言课程设计》日志（二）

4.4 库存回滚式更新

销售与进货记录的修改不能只改账册字段，还必须同步调整库存。以销售修改为例，系统先把旧销售数量加回库存，再检查新数量是否可行；如果可行，则扣减新数量并保存。如果新数量超过可用库存，则恢复原状态并返回错误。该算法比直接计算差值更容易理解，也更便于处理异常分支。

4.5 实时聚合统计

统计模块不保存冗余汇总文件，而是在收到请求时遍历数组。销售排行榜按 `product_id` 分组累加数量和金额，再进行降序排序；近 7 天统计使用日期字符串和本地时间计算起止范围；库存全景报告对每个商品遍历销售、进货记录并累计。该方案时间复杂度较低，适合当前数据规模。

5 系统设计中使用的 C 语言知识点

5.1 结构体、typedef 与数组

结构体用于把一条业务记录中的多个字段组织为一个整体。基本语法如下：

知识点	语法示例	在项目中的作用
结构体定义	<pre>typedef struct { char id[64]; int stock; double price; } Product;</pre>	定义商品的数据模型
结构体数组	<pre>Product products[MAX_PRODUCTS];</pre>	在内存中保存多条商品记录
成员访问	<pre>products[i].stock -= quantity;</pre>	读取或修改某个商品的库存字段
结构体指针	<pre>Product *p = &products[product_count];</pre>	直接填写数组末尾的新商品

5.2 指针、字符串与字符数组

C 语言字符串本质上是以 `'\0'` 结束的字符数组。项目中大量使用 `char *`、`const char *` 和字符数组处理路径、查询参数、JSON 以及文本文件字段。例如 `strchr(path, '?')` 用于寻找查询字符串起点，`strstr()` 用于模糊搜索，`strcmp()` 用于精确比较，`strncpy()` 用于限制复制长度。使用 `strncpy()` 后还要手动写入末尾 `'\0'`，避免源字符串过长时没有结束符。

5.3 函数、头文件与模块化

2026 春《C 语言课程设计》日志（二）

项目将声明放入 .h 文件，将实现放入 .c 文件，并通过 #include 引用。以数据层为例，data.h 中使用 `extern Product products[MAX_PRODUCTS]`；声明全局数组，data.c 中再提供实际定义。业务模块只需包含 data.h，就可以访问统一数据。函数采用“输入参数 + 输出缓冲区”的形式，例如 `void handle_get_products(const char *query, char *response_body);`，既适合 C 语言，也便于服务器路由统一调用。

5.4 文件读写

文本持久化使用标准文件 I/O。fopen(path, "r") 以只读模式打开文件，fgets(line, sizeof(line), fp) 逐行读取，fprintf(fp, "...", value) 格式化写出，fclose(fp) 释放文件资源。读取时先跳过表头，再将每一行解析为结构体；保存时先写表头，再遍历数组输出记录。

5.5 条件判断、循环与排序

if、else、for 和 while 构成主要控制流程。商品查询通过 for 循环遍历数组，再用多个 if 判断筛选条件；删除记录时使用 for 循环将后续元素向前移动；销售排行榜先聚合，再使用 qsort() 排序。判断库存时必须把失败分支放在修改数据之前，例如 `if (products[pidx].stock < quantity) { 返回错误; }`。

5.6 缓冲区、格式化输出与安全意识

C 语言需要开发者主动管理缓冲区。项目中使用 snprintf() 和 strncpy() 限制写入长度，使用 JsonBuilder.cap 与 JsonBuilder.len 记录可用空间，并在助手模块中用 SecureZeroMemory() 清除密钥。虽然课程设计规模较小，但仍要注意数组越界、字符串未终止、文件打开失败和网络请求超时等问题。

5.7 Socket 与 Windows API 调用

服务器通过 Winsock2 完成网络通信，典型顺序为 WSStartup() → socket() → bind() → listen() → accept() → recv()/send() → closesocket() → WSACleanup()。白泽助手则使用 WinHTTP 发起 HTTPS 请求。启动器使用 CreateProcessW() 隐藏启动后端，并通过 ShellExecuteW() 打开浏览器。这些调用说明 C 语言不仅能处理算法，也能直接使用操作系统能力构建完整应用。

2026 春《C 语言课程设计》日志（二）

6 阶段成果与后续完善

当前版本已经实现从启动、录入、查询、统计到关闭的完整演示流程。项目目录中还包含便携资源检查、启动器检查、关闭接口检查、前端视觉契约检查、生成数据完整性检查、白泽助手接口检查和销售草稿解析检查等测试脚本，用于降低功能调整带来的回归风险。

后续可从三个方向继续完善：第一，将文本文件升级为 SQLite，使用事务解决异常断电或中途写入失败问题；第二，将单线程服务器扩展为线程池模型，提高多用户访问能力；第三，增加供应商、会员、利润核算、条码扫描和报表导出功能，使系统更接近真实经营场景。

7 小组分工

日志（一）中的分工在实现阶段得到延续。随着便携启动、离线资源和白泽助手加入，部分集成功能由小组协作完成。

成员	主要负责内容	对应文件或模块
张子浩	数据层定义、文本持久化、演示数据生成	data.c、data.h、generate_data.c
李成俊	工具层、JSON 构建器、商品库存管理	utils.c、utils.h、product.c、product.h
张翔	销售、进货库存联动与前端交互	sale.c、purchase.c、frontend
杨宇辰	HTTP 服务器、统计分析与系统集成	server.c、stats.c、launcher.c
小组协作	便携化交付、白泽助手、联调与测试	assistant.c、assistant.jsx、tests

组员（手写签名）：
杨宇辰 张翔
张子浩 李成俊